Cosmos EVM 고급 기능 및 최적화

1. 고급 EVM 기능

1.1 커스텀 오프코드 (Custom Opcodes)

Cosmos EVM은 표준 이더리움 오프코드 외에 커스텀 오프코드를 추가할 수 있습니다.

```
// x/vm/types/opcodes.go
const (
// 커스텀 오프코드 정의
COSMOS_CALL = 0xf0 + iota
COSMOS_STATICCALL
COSMOS_DELEGATECALL
)


// 커스텀 오프코드 구현
func  opCosmosCall(pc *uint64, interpreter *vm.EVMInterpreter, scope *vm.ScopeCo
// 스택에서 파라미터 추출
addr := scope.Stack.pop()
inputOffset := scope.Stack.pop()
inputSize := scope.Stack.pop()
// Cosmos 네이티브 함수 호출
```

```
result, err := interpreter.evm.CosmosKeeper.CallNativeFunction(
common.BigToAddress(addr),
scope.Memory.GetPtr(int64(inputOffset.Uint64()), int64(inputSize.Uint64())),
)
if err != nil {
scope.Stack.push(big.NewInt(0)) // 실패
return nil, nil
}
scope.Stack.push(big.NewInt(1)) // 성공
return result, nil
}
```

1.2 EIP 구현 상태

```
// 지원되는 EIP 목록
var SupportedEIPs = map[int]bool{
155: true, // Simple replay attack protection
158: true, // State clearing
160: true, // EXP cost increase
170: true, // Contract code size limit
196: true, // Precompiled contracts for addition and scalar multiplication on th
197: true, // Precompiled contracts for optimal ate pairing check on the ellipti
```

```
198: true, // Precompiled contract for bigint modular exponentiation
211: true, // New opcodes: RETURNDATASIZE and RETURNDATACOPY
214: true, // New opcode STATICCALL
658: true, // Embedding transaction status code in receipts
1014: true, // Skinny CREATE2
1052: true, // EXTCODEHASH opcode
1283: true, // Net gas metering for SSTORE without dirty maps
1344: true, // ChainID opcode
1884: true, // Repricing for trie-size-dependent opcodes
2028: true, // Transaction data gas cost reduction
2200: true, // Structured Definitions for Net Gas Metering
2929: true, // Gas cost increases for state access opcodes
2930: true, // Optional access lists
3198: true, // BASEFEE opcode
3529: true, // Reduction in refunds
3541: true, // Reject new contracts starting with the 0xEF byte
3554: true, // Difficulty Bomb Delay to December 2021
}
```

1.3 동적 수수료 시장 (EIP-1559)

```go
// x/feemarket/keeper/keeper.go
func (k Keeper) CalculateBaseFee(ctx sdk.Context) *big.Int {
```

```go
params := k.GetParams(ctx)

// 이전 블록 정보 가져오기

prevBlock := ctx.BlockHeight() - 1

prevGasUsed := k.GetBlockGasUsed(ctx, prevBlock)

prevGasLimit := k.GetBlockGasLimit(ctx, prevBlock)

// EIP-1559 알고리즘 적용

baseFee := k.GetBaseFee(ctx)

if  prevGasUsed == 0 {

return  baseFee

}

// 가스 사용률 계산

gasUsedRatio := new(big.Int).Div(

new(big.Int).Mul(prevGasUsed, big.NewInt(100)),

prevGasLimit,

)

// Base fee 조정

if  gasUsedRatio.Cmp(big.NewInt(50)) > 0 {

// 50% 이상 사용 시 증가

increase := new(big.Int).Div(

new(big.Int).Mul(baseFee, new(big.Int).Sub(gasUsedRatio, big.NewInt(50))),

new(big.Int).Mul(big.NewInt(50), params.BaseFeeChangeDenominator),

)

baseFee = new(big.Int).Add(baseFee, increase)
```

```go
} else {
// 50% 미만 사용 시 감소
decrease := new(big.Int).Div(
new(big.Int).Mul(baseFee, new(big.Int).Sub(big.NewInt(50), gasUsedRatio)),
new(big.Int).Mul(big.NewInt(50), params.BaseFeeChangeDenominator),
)
baseFee = new(big.Int).Sub(baseFee, decrease)
}
// 최소값 보장
if baseFee.Cmp(params.MinBaseFee) < 0 {
baseFee = params.MinBaseFee
}
return baseFee
}
```

2. 성능 최적화

2.1 상태 캐싱

```go
// x/vm/statedb/cache.go
type StateCache struct {
// 계정 캐시
accounts map[common.Address]*Account
```

```go
// 스토리지 캐시
storage  map[common.Address]map[common.Hash]common.Hash
// 코드 캐시
codes  map[common.Address][]byte
// 캐시 통계
hits  uint64
misses  uint64
}

func (sc *StateCache) GetAccount(addr common.Address) (*Account, bool) {
account, exists := sc.accounts[addr]
if  exists {
atomic.AddUint64(&sc.hits, 1)
return  account, true
}
atomic.AddUint64(&sc.misses, 1)
return  nil, false
}

func (sc *StateCache) SetAccount(addr common.Address, account *Account) {
sc.accounts[addr] = account
}
```

2.2 병렬 트랜잭션 처리

```go
// mempool/parallel.go
type ParallelExecutor struct {
workers int
txQueue chan *Transaction
resultChan chan *ExecutionResult
}

func (pe *ParallelExecutor) ExecuteBlock(txs []*Transaction) ([]*ExecutionResult
// 의존성 분석
dependencies := pe.analyzeDependencies(txs)
// 병렬 실행 가능한 트랜잭션 그룹화
groups := pe.groupIndependentTxs(txs, dependencies)
var results []*ExecutionResult
for _, group := range groups {
// 각 그룹을 병렬로 실행
groupResults := pe.executeGroup(group)
results = append(results, groupResults...)
}
return results, nil
}
```

```go
func (pe *ParallelExecutor) analyzeDependencies(txs []*Transaction) map[int][]in

dependencies := make(map[int][]int)

addressAccess := make(map[common.Address][]int)

for i, tx := range txs {

// 트랜잭션이 접근하는 주소들 분석

addresses := pe.getAccessedAddresses(tx)

for _, addr := range addresses {

// 이전에 같은 주소에 접근한 트랜잭션들과 의존성 생성

if prevTxs, exists := addressAccess[addr]; exists {

dependencies[i] = append(dependencies[i], prevTxs...)

}

addressAccess[addr] = append(addressAccess[addr], i)

}

}

return dependencies

}
```

2.3 메모리 풀 최적화

```go
// mempool/priority.go

type PriorityMempool struct {

// 가스 가격별 우선순위 큐

priceHeap *GasPriceHeap
```

```go
// 논스별 트랜잭션 맵
nonceMap  map[common.Address]*TxList
// 전체 트랜잭션 수
count  int
// 최대 크기
maxSize  int
}


func (pm *PriorityMempool) Add(tx *Transaction) error {
// 메모리 풀 크기 확인
if  pm.count >= pm.maxSize {
// 가장 낮은 가스 가격의 트랜잭션 제거
if !pm.evictLowestPriceTx() {
return  ErrMempoolFull
}
}
// 논스 검증
if  err := pm.validateNonce(tx);  err != nil {
return  err
}
// 트랜잭션 추가
pm.priceHeap.Push(tx)
pm.addToNonceMap(tx)
```

```go
pm.count++
return nil
}

func (pm *PriorityMempool) Select(maxGas uint64) []*Transaction {
var selected []*Transaction
var totalGas uint64
// 가스 가격 순으로 트랜잭션 선택
for pm.priceHeap.Len() > 0 && totalGas < maxGas {
tx := pm.priceHeap.Peek()
if totalGas+tx.Gas() > maxGas {
break
}
// 논스 순서 확인
if pm.isExecutable(tx) {
selected = append(selected, tx)
totalGas += tx.Gas()
pm.priceHeap.Pop()
} else {
break
}
}
return selected
```

```
}
```

3. 고급 보안 기능

3.1 권한 기반 EVM

```
// x/vm/keeper/permissions.go
type PermissionKeeper struct {
// 계정별 권한
permissions  map[common.Address]Permission
// 컨트랙트별 권한
contractPermissions  map[common.Address]Permission
}

type Permission struct {
CanCreate  bool  // 컨트랙트 생성 권한
CanCall  bool  // 컨트랙트 호출 권한
CanDeploy  bool  // 특정 컨트랙트 배포 권한
Whitelist []common.Address // 호출 가능한 컨트랙트 목록
Blacklist []common.Address // 호출 금지된 컨트랙트 목록
}

func (pk *PermissionKeeper) CheckCreatePermission(creator common.Address) error
```

```go
perm, exists := pk.permissions[creator]
if !exists {
return ErrNoPermission
}
if !perm.CanCreate {
return ErrCreateNotAllowed
}
return nil
}


func (pk *PermissionKeeper) CheckCallPermission(caller, contract common.Address)
perm, exists := pk.permissions[caller]
if !exists {
return ErrNoPermission
}
if !perm.CanCall {
return ErrCallNotAllowed
}
// 블랙리스트 확인
for _, blocked := range perm.Blacklist {
if blocked == contract {
return ErrContractBlocked
}
```

```
}
// 화이트리스트가 있는 경우 확인
if len(perm.Whitelist) > 0 {
allowed := false
for _, allowed := range perm.Whitelist {
if allowed == contract {
allowed = true
break
}
}
if !allowed {
return ErrContractNotWhitelisted
}
}
return nil
}
```

3.2 가스 한도 관리

```
// x/vm/keeper/gas.go
type GasManager struct {
// 계정별 가스 한도
accountLimits map[common.Address]uint64
```

```
// 컨트랙트별 가스 한도
contractLimits  map[common.Address]uint64
// 글로벌 가스 한도
globalLimit  uint64
}


func (gm *GasManager) CheckGasLimit(caller, contract common.Address, gas  uint64
// 글로벌 한도 확인
if  gas > gm.globalLimit {
return  ErrGasLimitExceeded
}
// 계정별 한도 확인
if  limit, exists := gm.accountLimits[caller];  exists {
if  gas > limit {
return  ErrAccountGasLimitExceeded
}
}
// 컨트랙트별 한도 확인
if  contract != (common.Address{}) {
if  limit, exists := gm.contractLimits[contract];  exists {
if  gas > limit {
return  ErrContractGasLimitExceeded
}
```

```
}
}
return nil
}
```

4. 고급 디버깅 도구

4.1 상세 트레이싱

```
// x/vm/tracer/detailed.go
type DetailedTracer struct {
// 실행 스택
callStack []*CallFrame
// 상태 변경 추적
stateChanges []StateChange
// 가스 사용량 추적
gasUsage []GasUsage
// 이벤트 로그
events []Event
}

type CallFrame struct {
Type string `json:"type"`
```

```
From common.Address `json:"from"`

To common.Address `json:"to"`

Input []byte  `json:"input"`

Gas  uint64  `json:"gas"`

GasUsed  uint64  `json:"gasUsed"`

Output []byte  `json:"output"`

Error  string  `json:"error,omitempty"`

Calls []*CallFrame `json:"calls,omitempty"`

}



func (dt *DetailedTracer) CaptureStart(env *vm.EVM, from common.Address, to comm

frame := &CallFrame{

Type: "CALL",

From: from,

To: to,

Input: input,

Gas: gas,

}

if  create {

frame.Type = "CREATE"

}

dt.callStack = append(dt.callStack, frame)

}
```

```go
func (dt *DetailedTracer) CaptureEnd(output []byte, gasUsed  uint64, t time.Dura

if  len(dt.callStack) == 0 {

return

}

frame := dt.callStack[len(dt.callStack)-1]

frame.Output = output

frame.GasUsed = gasUsed

if  err != nil {

frame.Error = err.Error()

}

}
```

4.2 성능 프로파일링

```go
// x/vm/profiler/profiler.go

type EVMProfiler struct {

// 오프코드별 실행 시간

opcodeStats  map[vm.OpCode]*OpcodeStat

// 함수별 실행 시간

functionStats  map[string]*FunctionStat

// 메모리 사용량

memoryUsage []MemorySnapshot
```

```go
}

type OpcodeStat struct {
Count  uint64  `json:"count"`
TotalTime time.Duration `json:"total_time"`
AverageTime time.Duration `json:"average_time"`
MaxTime time.Duration `json:"max_time"`
}

func (ep *EVMProfiler) CaptureState(env *vm.EVM, pc  uint64, op vm.OpCode, gas,
start := time.Now()
// 오프코드 실행 시간 측정
defer  func() {
elapsed := time.Since(start)
stat, exists := ep.opcodeStats[op]
if !exists {
stat = &OpcodeStat{}
ep.opcodeStats[op] = stat
}
stat.Count++
stat.TotalTime += elapsed
stat.AverageTime = stat.TotalTime / time.Duration(stat.Count)
if  elapsed > stat.MaxTime {
```

```
stat.MaxTime = elapsed

}

}()

// 메모리 사용량 스냅샷

if len(ep.memoryUsage) < 1000 { // 최대 1000개 스냅샷

ep.memoryUsage = append(ep.memoryUsage, MemorySnapshot{

PC: pc,

OpCode: op,

Memory: len(scope.Memory.Data()),

Stack: scope.Stack.len(),

})

}

}
```

5. 고급 거버넌스 기능

5.1 동적 파라미터 업데이트

```
// x/vm/keeper/governance.go

func (k Keeper) HandleParameterChangeProposal(ctx sdk.Context, proposal *types.P

for _, change := range proposal.Changes {

switch change.Key {

case "evm_denom":
```

```
k.SetEvmDenom(ctx, change.Value)
case "enable_create":
enable, err := strconv.ParseBool(change.Value)
if err != nil {
return err
}
k.SetEnableCreate(ctx, enable)
case "enable_call":
enable, err := strconv.ParseBool(change.Value)
if err != nil {
return err
}
k.SetEnableCall(ctx, enable)
case "extra_eips":
var eips []int64
if err := json.Unmarshal([]byte(change.Value), &eips); err != nil {
return err
}
k.SetExtraEIPs(ctx, eips)
default:
return fmt.Errorf("unknown parameter: %s", change.Key)
}
}
```

```
return nil
}
```

5.2 업그레이드 메커니즘

```
// x/vm/keeper/upgrade.go
func (k Keeper) HandleSoftwareUpgradeProposal(ctx sdk.Context, proposal *upgrade
// 업그레이드 계획 설정
plan := upgradetypes.Plan{
Name: proposal.Plan.Name,
Height: proposal.Plan.Height,
Info: proposal.Plan.Info,
}
// 업그레이드 핸들러 등록
k.upgradeKeeper.SetUpgradeHandler(plan.Name, func(ctx sdk.Context, plan upgradet
// EVM 관련 업그레이드 로직
return k.RunMigrations(ctx, fromVM)
})
return k.upgradeKeeper.ScheduleUpgrade(ctx, plan)
}
```

이 문서는 Cosmos EVM의 고급 기능과 최적화 방법을 상세히 설명합니다. 이로써 Cosmos EVM 구현체의 전체적인 분석이 완료되었습니다.